SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design
Weightage: 20%

QUESTIONS: **Total Marks: 25**

1. A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.

2. An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.

3. A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.

4.An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.
5.A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Code of Conduct:

*I **G.MOHAMMAD SHAREEF/(192311288)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:G.MOHAMMAD SHAREEF

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection
Feasibility	10%	Applies relevant concepts ensuring high feasibility and practicality	Applies concepts with minor issues	Limited feasibility or incorrect application	Solution is impractical or conceptually incorrect

1. REST vs SOAP – Web Service Selection

REST vs SOAP

REST	SOAP
Uses HTTP methods	XML-based messaging protocol
Usually uses JSON	Mainly uses XML
Lightweight and fast	More complex
Easy for mobile/web applications	Suitable for enterprise systems
Highly scalable	Reliable but heavier
Simple API development	Strong standards for security and transactions

Selected Approach: REST

For a mobile and web application, REST API is the better choice.

Design

Mobile App / Web App

↓

HTTPS

↓

REST API

↓

Business Logic

↓

Database

Example:

GET /api/products

POST /api/users

PUT /api/users/101

DELETE /api/users/101

Justification

- **JSON responses are lightweight.**
- **Faster communication than SOAP in typical web/mobile scenarios.**
- **Stateless architecture supports horizontal scaling.**
- **HTTPS, authentication tokens, and authorization provide security.**
- **Easy integration with mobile and web applications.**

2. Scalable Service Architecture

Basic Architecture

Users

↓

Mobile / Web Apps

↓

Load Balancer

↓

API Server

↓

Business Logic

↓

Database

The load balancer distributes requests among multiple servers.

Alternative Scalable Architecture

Users

↓

CDN

↓

Load Balancer

↓

API Gateway

↓

Microservices
├── User Service
├── Order Service
└── Payment Service
↓
Cache / Message Queue
↓
Database Cluster
Advantages

- **Multiple servers handle requests.**
- **Load balancing prevents one server from becoming overloaded.**
- **Caching reduces database requests.**
- **Message queues handle background tasks.**
- **Database replication improves availability.**
- **Services can be scaled independently.**

Conclusion: The second architecture is more suitable for a high-traffic platform because individual components can be scaled independently.

3. Secure Banking Communication
Available Protocols

- **HTTP: Not encrypted; unsuitable for banking.**
- **HTTPS: HTTP combined with TLS encryption.**
- **FTP: Mainly for file transfer.**
- **SMTP: Used for email communication.**
- **WebSocket over TLS (WSS): Useful for secure real-time communication.**

Selected Protocol: HTTPS
Banking Client
↓
HTTPS
(TLS)
↓
Banking Server
↓
Database
Justification
Security:

- **Encrypts data during transmission.**
- **Protects passwords and financial information.**
- **Helps prevent man-in-the-middle attacks.**
- **Server certificates authenticate the server.**

Reliability:

- **Uses HTTP request/response mechanisms.**
- **Supports status codes and error handling.**

Performance:

- **Modern HTTP/2 and HTTP/3 can provide efficient communication.**
- **Connection reuse reduces overhead.**

Conclusion: HTTPS is essential for banking applications.

4. Third-Party API Integration

API Integration Strategy

Application

↓

API Client

↓

HTTPS Request

↓

Third-Party API

↓

JSON Response

↓

Validation

↓

Application

Steps

1. **Register with the third-party service.**
2. **Obtain API credentials.**
3. **Store credentials securely.**
4. **Send requests over HTTPS.**
5. **Authenticate using an API key or OAuth 2.0.**
6. **Validate the response.**
7. **Process the returned data.**
8. **Handle errors and timeouts.**
9. **Log important failures without storing sensitive information.**

Example:

GET /maps/location

Authorization: Bearer ACCESS_TOKEN

Alternative Approach

Use a backend integration layer instead of calling the third-party API directly from the client.

Mobile/Web App

↓

Your Backend

↓

Third-Party API

Advantages:

- **API credentials remain hidden.**
- **Responses can be cached.**
- **Errors can be retried.**

- **Third-party API changes can be isolated.**
- **Better monitoring and security.**

5. Error Handling and Fault Tolerance

Basic Error Handling

Client Request

↓

API Server

↓

Validate Request

↓

Process Request

↓

Success? ── Yes → Response

│

No

↓

Error Handler

↓

Error Response

Example:

```
{
  "status": 500,
  "message": "Service temporarily unavailable"
}
```

Fault-Tolerance Mechanisms

- **Timeouts: Prevent requests from waiting indefinitely.**
- **Retries: Retry temporary failures with exponential backoff.**
- **Circuit Breaker: Temporarily stops calls to a failing service.**
- **Fallback: Provides cached or alternative data when possible.**
- **Load Balancing: Distributes traffic across multiple servers.**
- **Health Checks: Detect unhealthy servers.**
- **Logging and Monitoring: Helps identify failures quickly.**

Alternative Strategies

Caching: Serve frequently requested data from cache.

Message Queues: Process non-critical tasks asynchronously.

Redundant Servers: Keep multiple instances available.

Graceful Degradation: Show limited functionality instead of completely failing.

Conclusion

A combination of timeouts, retries, circuit breakers, load balancing, monitoring, and graceful fallback provides better reliability and a smoother user experience.